

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 6: TRANSFORM-AND-CONQUER

(HEAPS - PROBLEM REDUCTION)

Abdullah Bal, PhD

Objectives



Recap

Representation Change

Multiway Search Trees

2-3 Trees



Heaps and Heapsort



Problem Reduction



LCM Optimization

Heaps

The data structure called the “heap” is definitely **not a disordered pile** of items as the word’s definition in a standard dictionary might suggest.

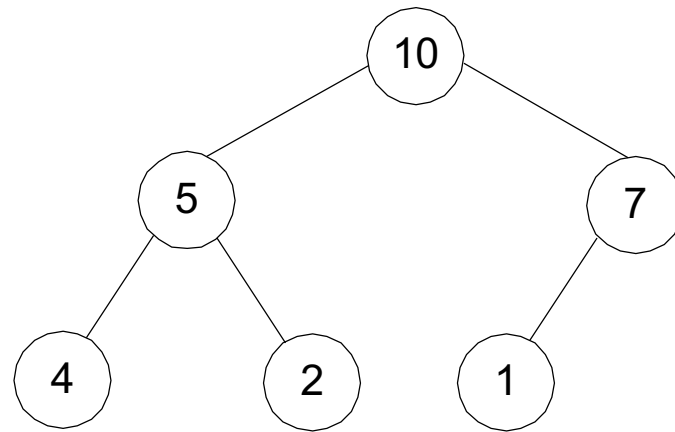
Rather, it is a clever, partially ordered data structure that is especially suitable for **implementing priority queues**.

- It is primarily an efficient implementation of these operations that makes the **heap both interesting and useful**.
- Priority queues arise naturally in such applications as **scheduling job executions** by computer operating systems and **traffic management by communication networks**.
- They also arise in several important algorithms, e.g., **Prim’s algorithm, Dijkstra’s algorithm, Huffman encoding, and branch-and-bound applications**.
- The heap is also the data structure that serves as a cornerstone of a theoretically important sorting algorithm called **heapsort**.

Heaps

Definition **A heap is a binary tree** with keys at its nodes (one key per node) such that:

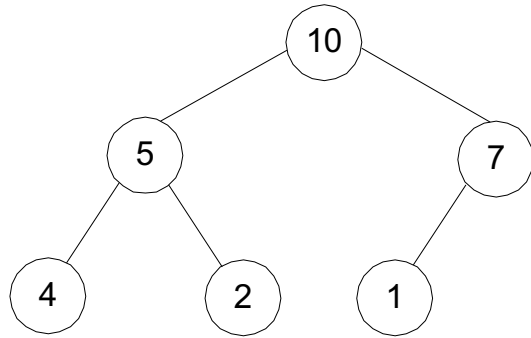
1. It is essentially complete, i.e., all its levels are full except possibly the last level, where only some rightmost keys may be missing



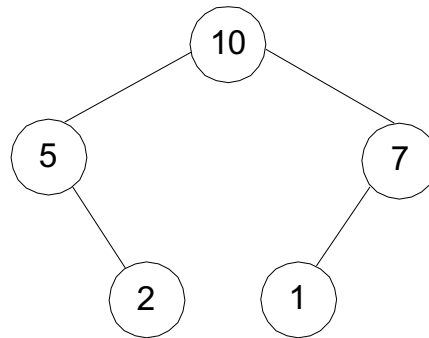
2. Max Heap: The key at each node is $\geq$ keys at its children

Min Heap: The key at each node is $\leq$ keys at its children

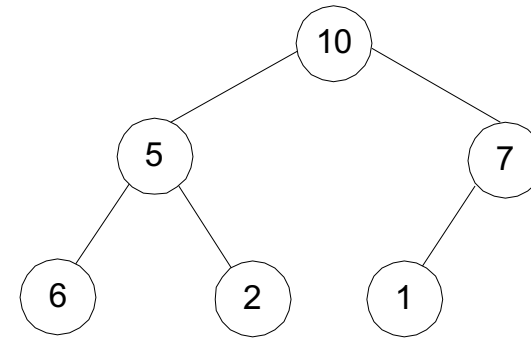
Illustration of the heap's definition



a heap



not a heap



not a heap

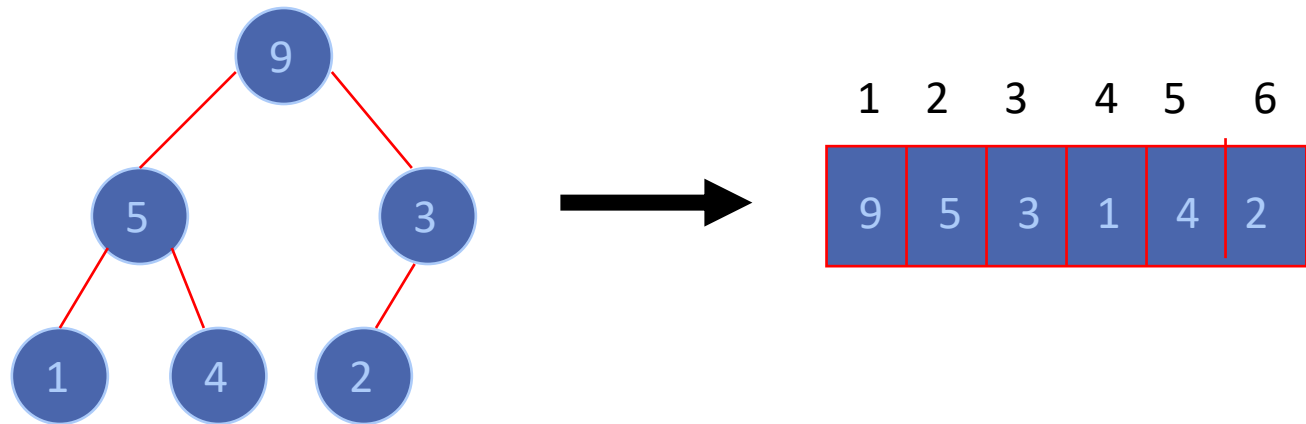
Note: Heap's elements are **ordered top down** (along any path down from its root), but they are **not ordered left to right**

Heap's Array Representation

- Store heap's elements in an array (whose elements indexed, for convenience, 1 to n) in top-down left-to-right order

- Example:

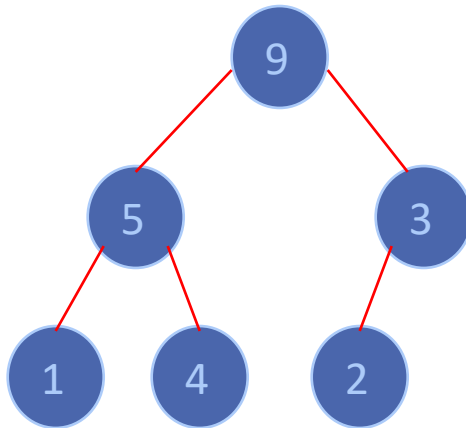
- Left child of node j is at $2j$
- Right child of node j is at $2j+1$
- Parent of node j is at floor $\lfloor j/2 \rfloor$ where j is an arbitrary index.



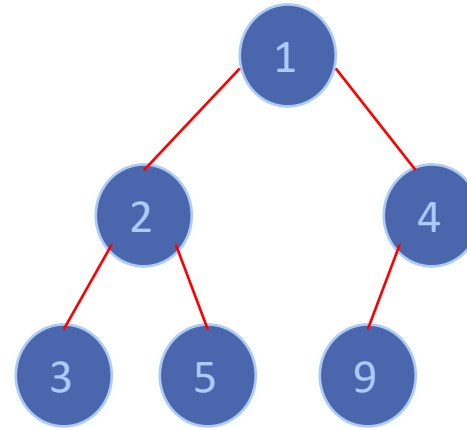
Some Important Properties of a Heap

- Given n , there exists a unique binary tree with n nodes that is essentially complete, with $h = \lfloor \log_2 n \rfloor$
- The root contains the largest key
- The subtree rooted at any node of a heap is also a heap
- A heap can be represented as an array

Max Heap and Min Heap



Max Heap

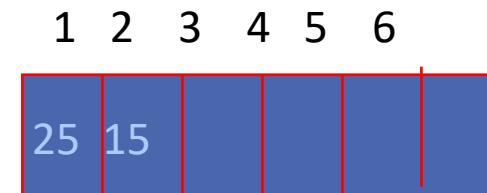
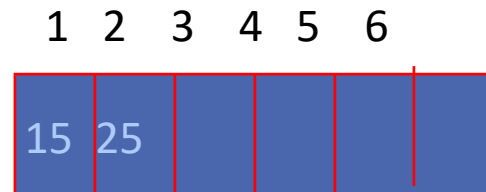
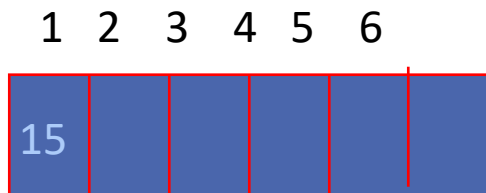
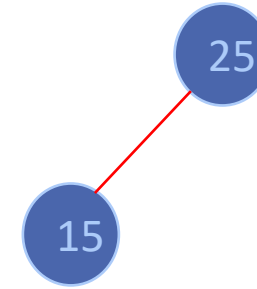
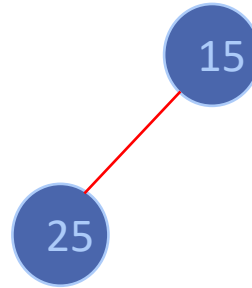


Min Heap

HEAP CREATION TOP DOWN

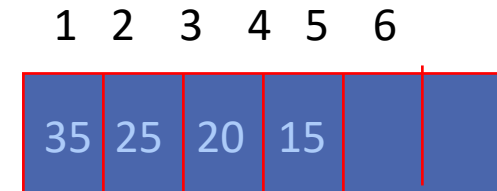
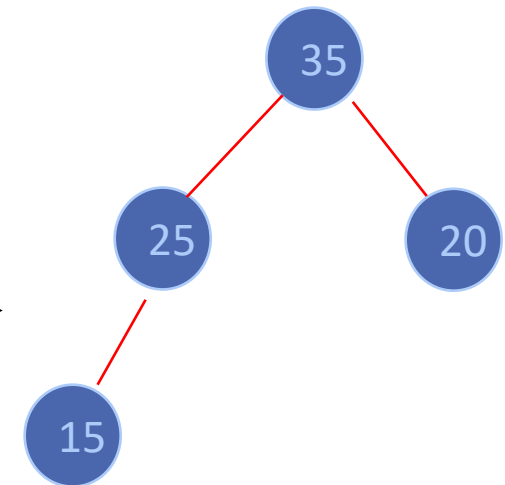
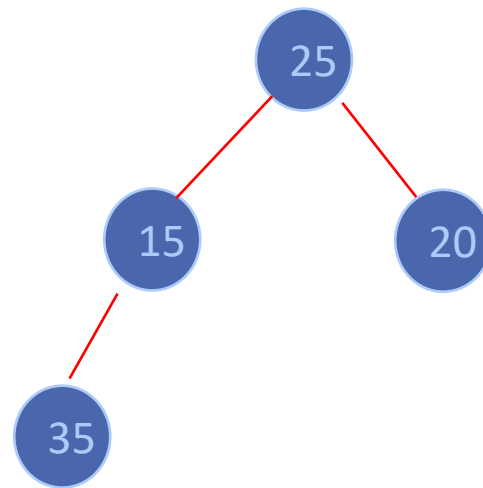
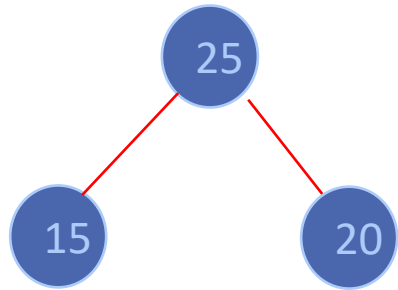
Example 1: Heap Creation (Top-Down)

Create a max heap for the list 15, 25, 20, 35, 45, 10



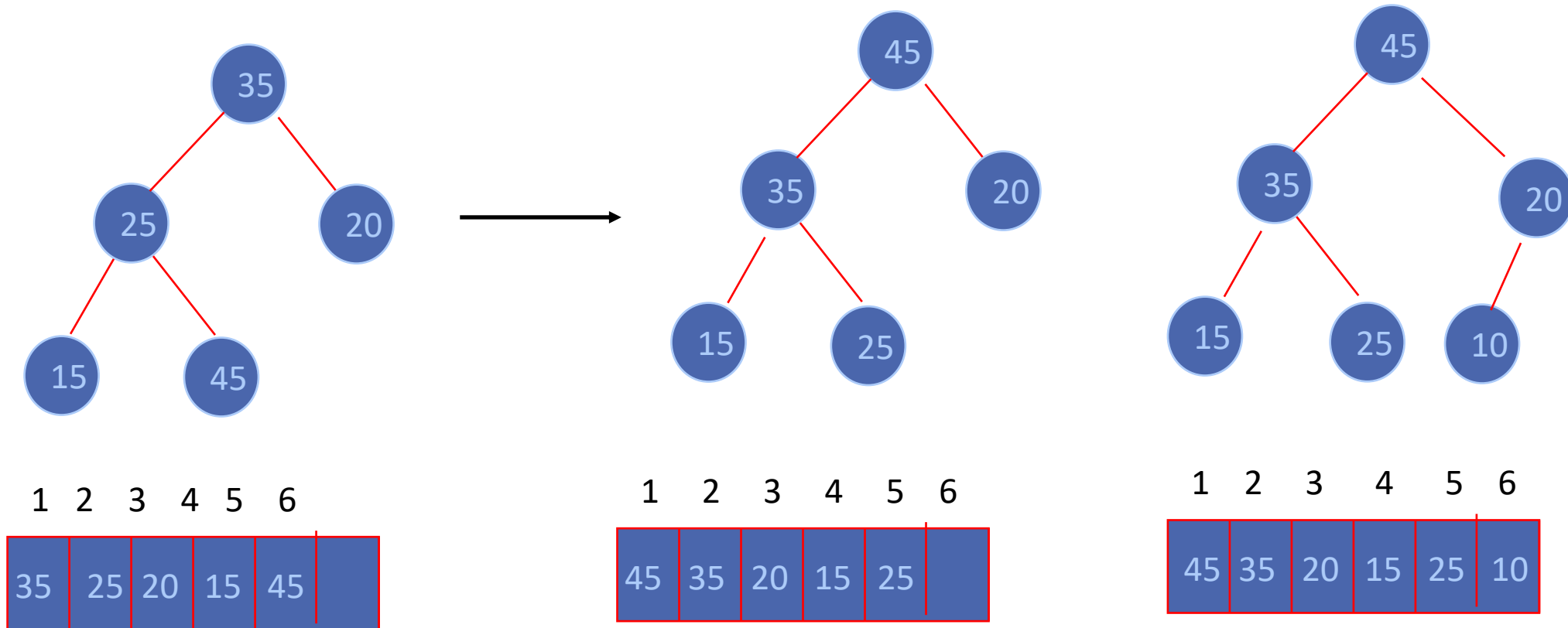
Example 1: Heap Creation (Top-Down)

Create a max heap for the list 15, 25, 20, 35, 45, 10



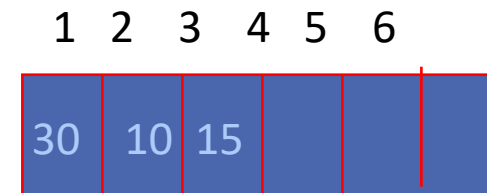
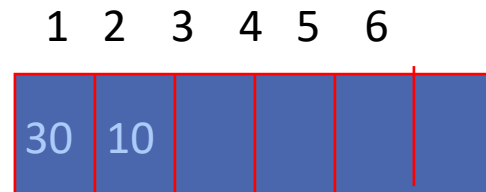
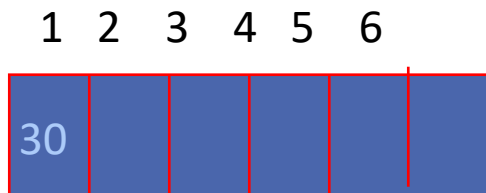
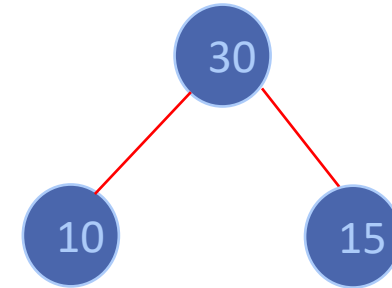
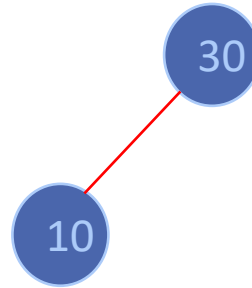
Example 1: Heap Creation (Top-Down)

Create a max heap for the list 15, 25, 20, 35, 45, 10



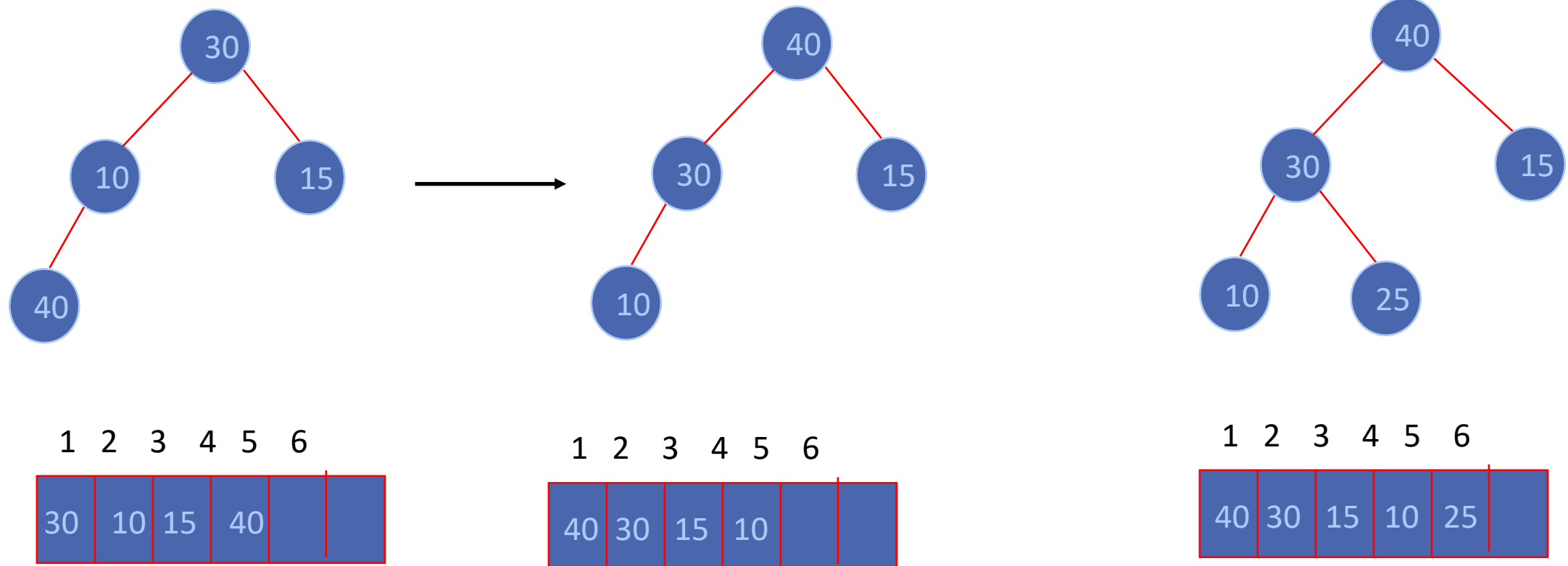
Example 2: Heap Creation (Top-Down)

Create a max heap for the list 30, 10, 15, 40, 25, 35



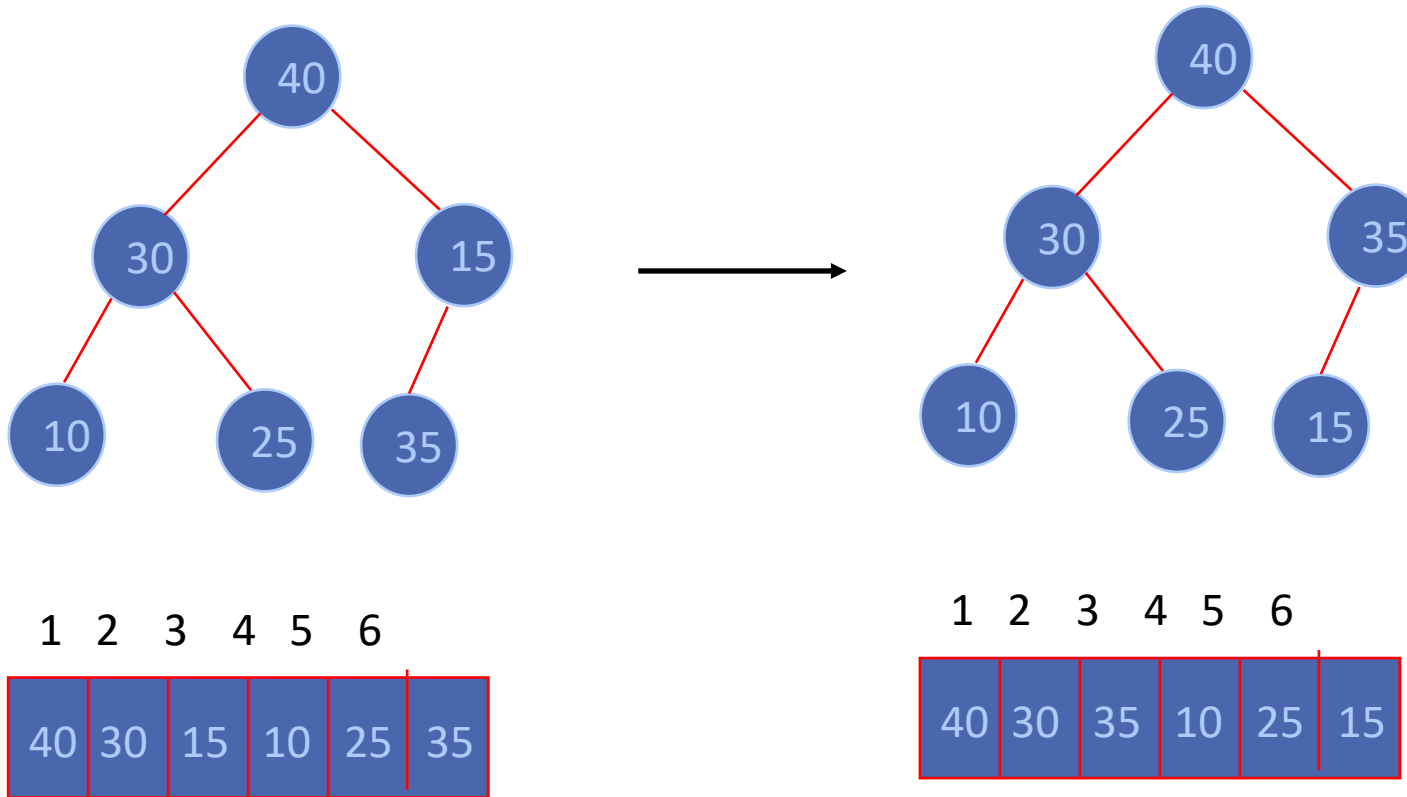
Example 2: Heap Creation (Top-Down)

Create a max heap for the list 30, 10, 15, 40, 25, 35



Example 2: Heap Creation (Top-Down)

Create a max heap for the list 30, 10, 15, 40, 25, 35



HEAP CREATION

BOTTOM UP

Heap Construction (Bottom-up)

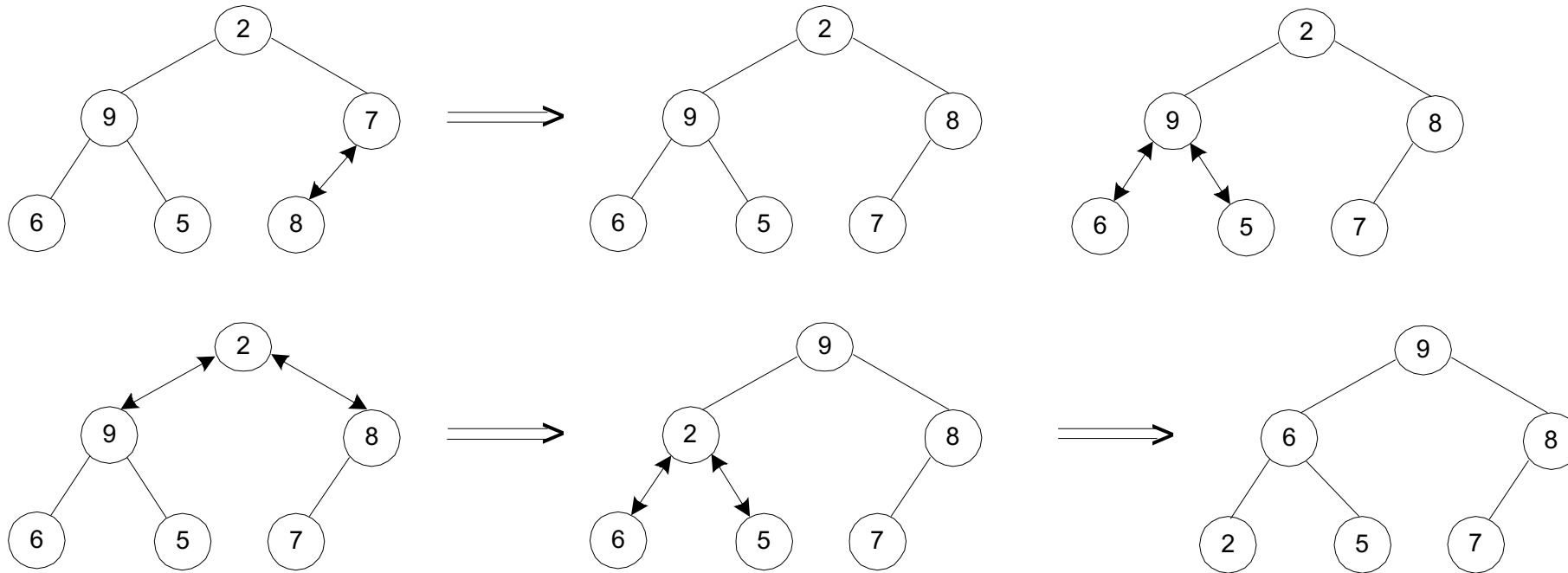
Step 0: Initialize the structure with keys in the order given

Step 1: Starting with the last (rightmost) parental node, fix the heap rooted at it,
if it doesn't satisfy the heap condition: keep exchanging it with its largest child until
the heap condition holds

Step 2: Repeat Step 1 for the preceding parental node

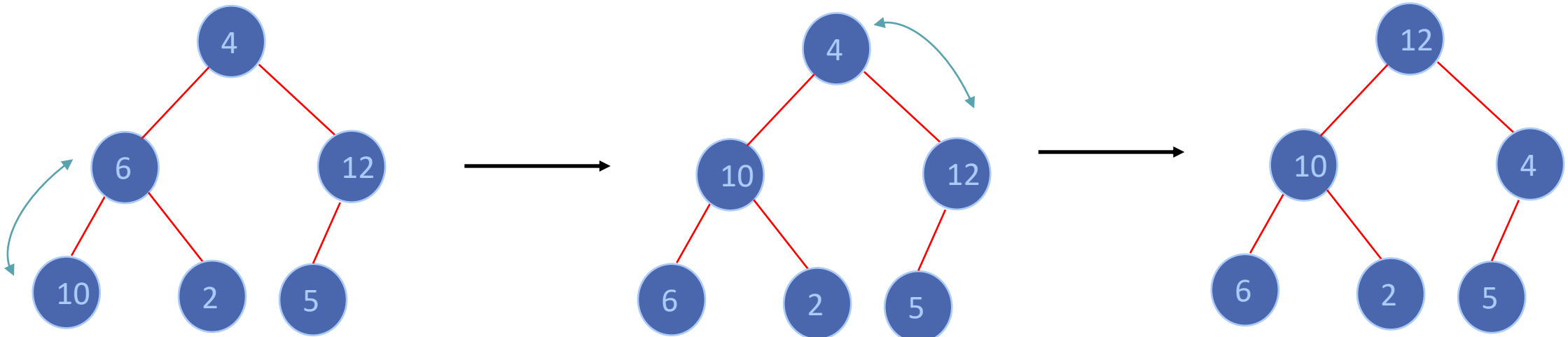
Example 1: Heap Construction (Bottom-up)

Construct a max heap for the list 2, 9, 7, 6, 5, 8



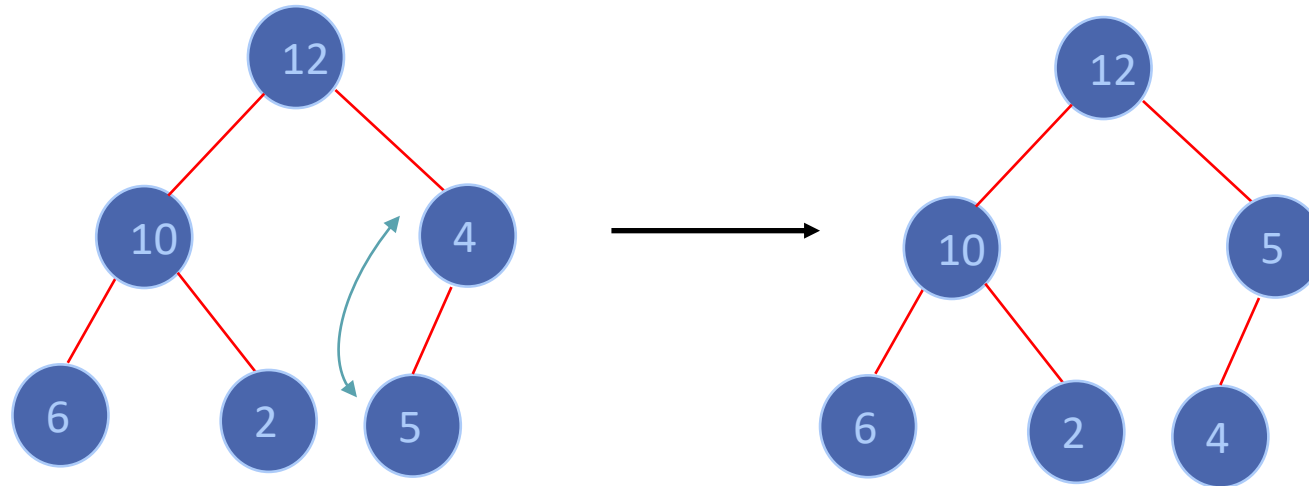
Example 2: Heap Construction (Bottom-up)

Construct a max heap for the list 4, 6, 12, 10, 2, 5



Example 2: Heap Construction (Bottom-up)

Construct a max heap for the list 4, 6, 12, 10, 2, 5



DELETION

Deletion

- Start to delete by max element of the heap which is the root's key in Max. Heap.

Step 1: Exchange the root's key with the last key K of the heap.

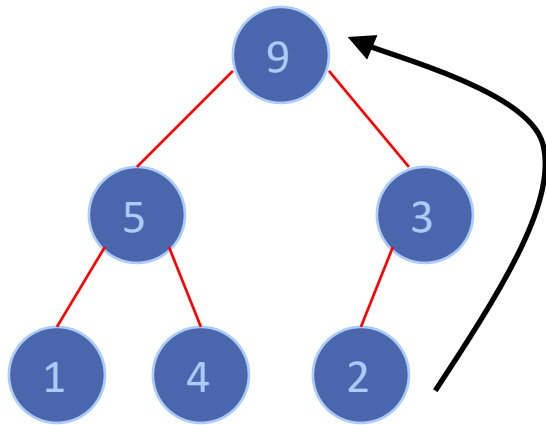
Step 2: Decrease the heap's size by 1.

Step 3: “Heapify” the smaller tree by sifting K down the tree exactly in the same way we did it in the bottom-up heap construction algorithm.

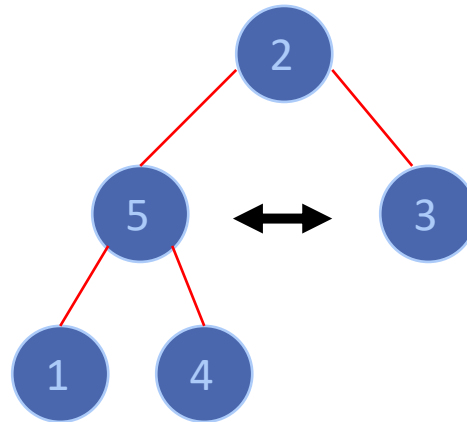
Deletion

- That is, verify the parental dominance for K: if it holds, we are done; if not, swap K with the larger of its children and repeat this operation until the parental dominance condition holds for K in its new position.
- The efficiency of deletion is determined by the number of key comparisons needed to “heapify” the tree after the swap has been made and the size of the tree is decreased by 1.
- Since this cannot require more key comparisons than twice the heap’s height, the time efficiency of deletion is in $O(\log n)$ as well.

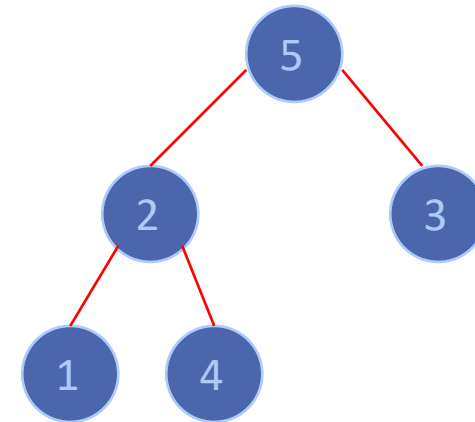
Deletion



1	2	3	4	5	6
9	5	3	1	4	2

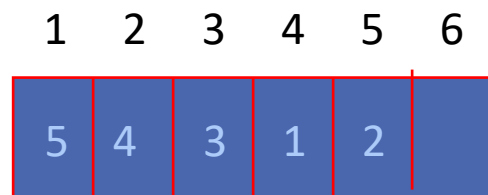
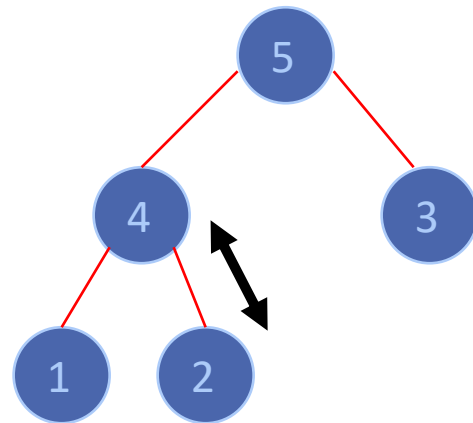


1	2	3	4	5	6
2	5	3	1	4	



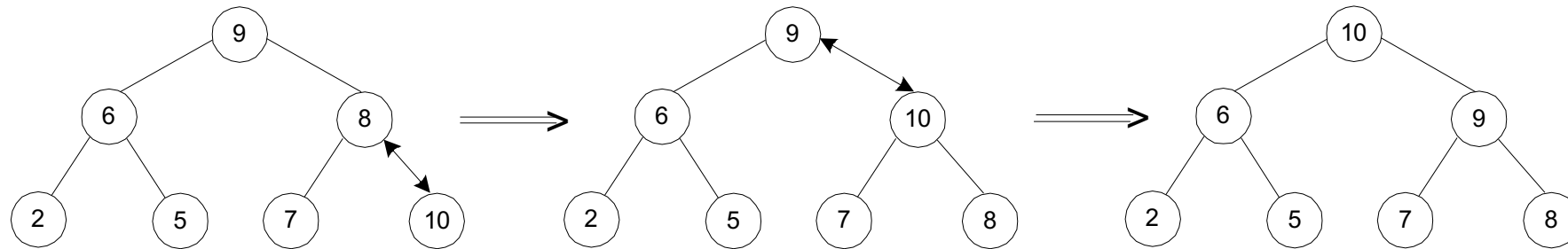
1	2	3	4	5	6
5	2	3	1	4	

Deletion



Insertion of a New Element into a Heap

- Insert the new element at last position in heap.
- Compare it with its parent and, if it violates heap condition, exchange them
- Continue comparing the new element with nodes up the tree until the heap condition is satisfied
- Example: Insert key 10



- Efficiency: $O(\log n)$

HEAPSORT

Heapsort

Stage 1: Construct a heap for a given list of n keys

Stage 2: Repeat operation of root removal $n-1$ times:

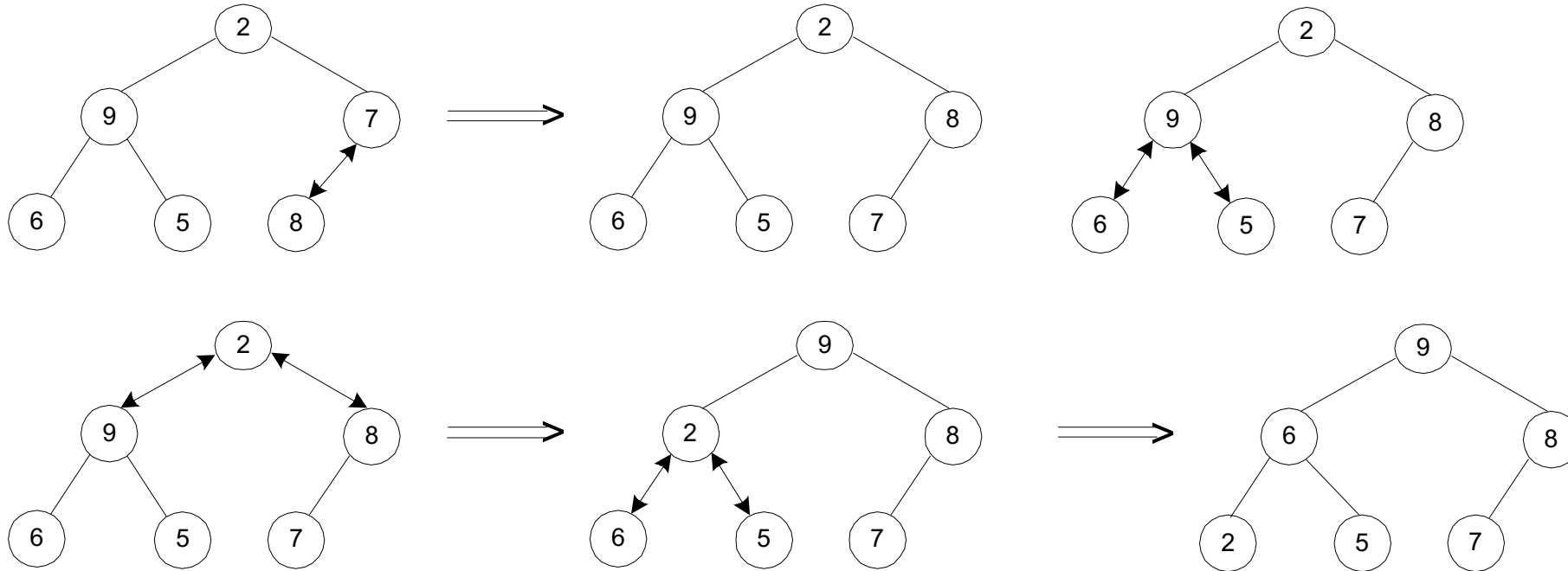
- Exchange keys in the root and in the last (rightmost) leaf
- Decrease heap size by 1
- If necessary, swap new root with larger child until the heap condition holds

Example 1: Sorting by Heapsort

Sort the list 2, 9, 7, 6, 5, 8 by heapsort

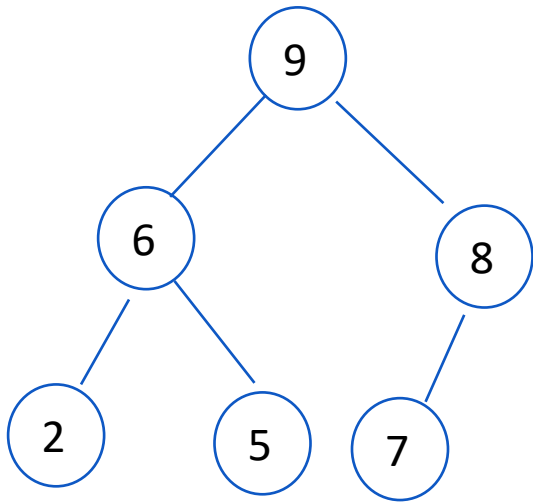
Stage 1: Heap Construction (Bottom-up)

Heap construction for 2, 9, 7, 6, 5, 8

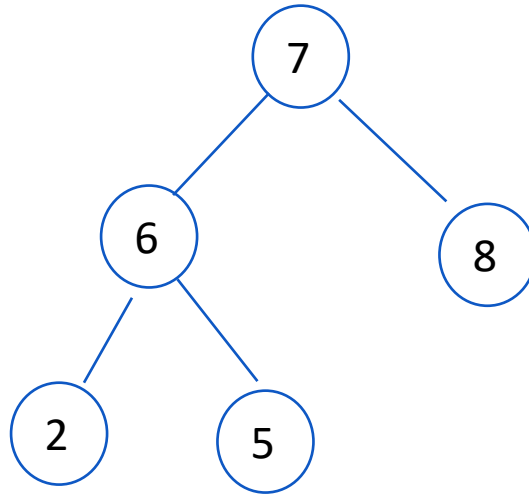


Constructed Heap: 9 6 8 2 5 7

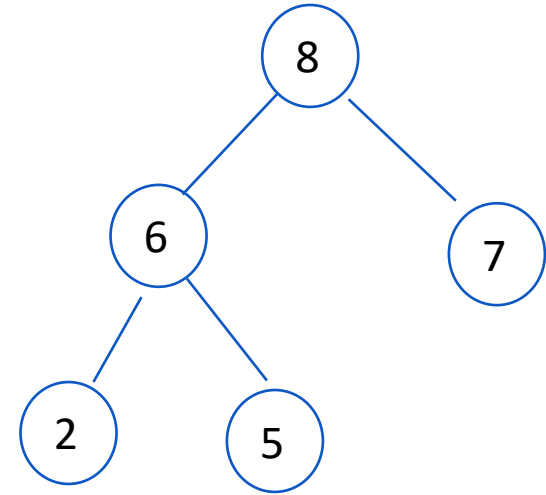
Stage 2: Root/Max Removal



9 6 8 2 5 7

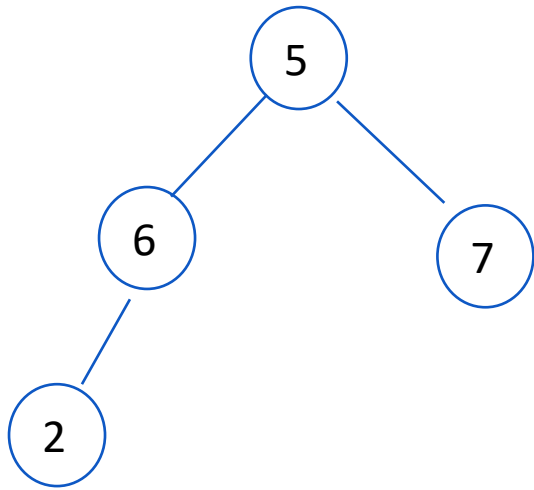


7 6 8 2 5 | 9

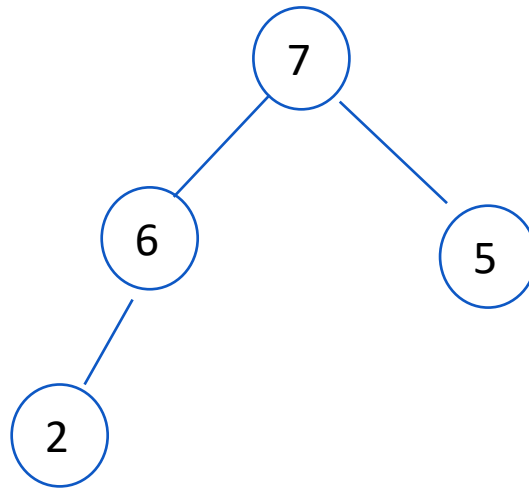


8 6 7 2 5 | 9

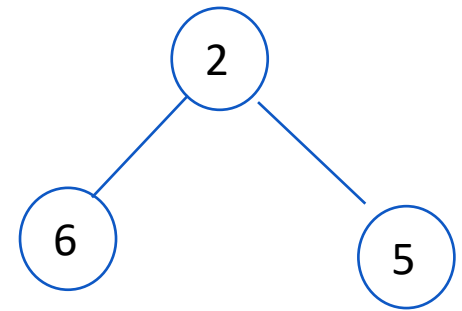
Stage 2: Root/Max Removal



5 6 7 2 | 8 9

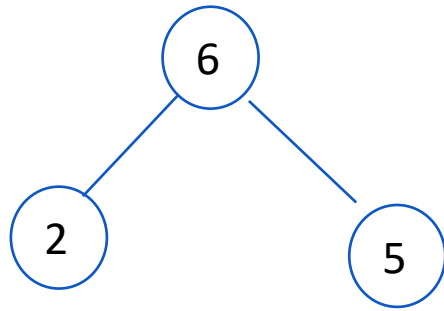


7 6 5 2 | 8 9

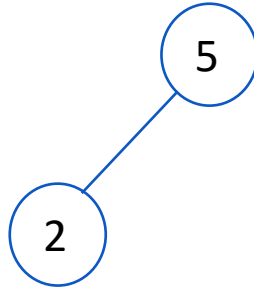


2 6 5 | 7 8 9

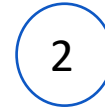
Stage 2: Root/Max Removal



6 2 5 | 7 8 9



2 5 | 6 7 8 9



2 | 5 6 7 8 9

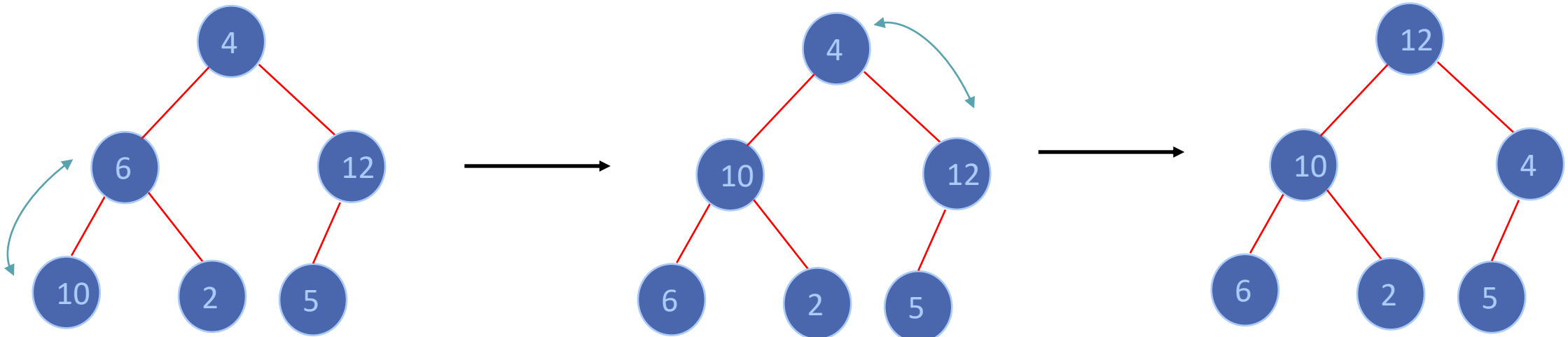
Sorted Array: 2 5 6 7 8 9

Example 2: Sorting by Heapsort

Sort the list 4, 6, 12, 10, 2, 5 by heapsort

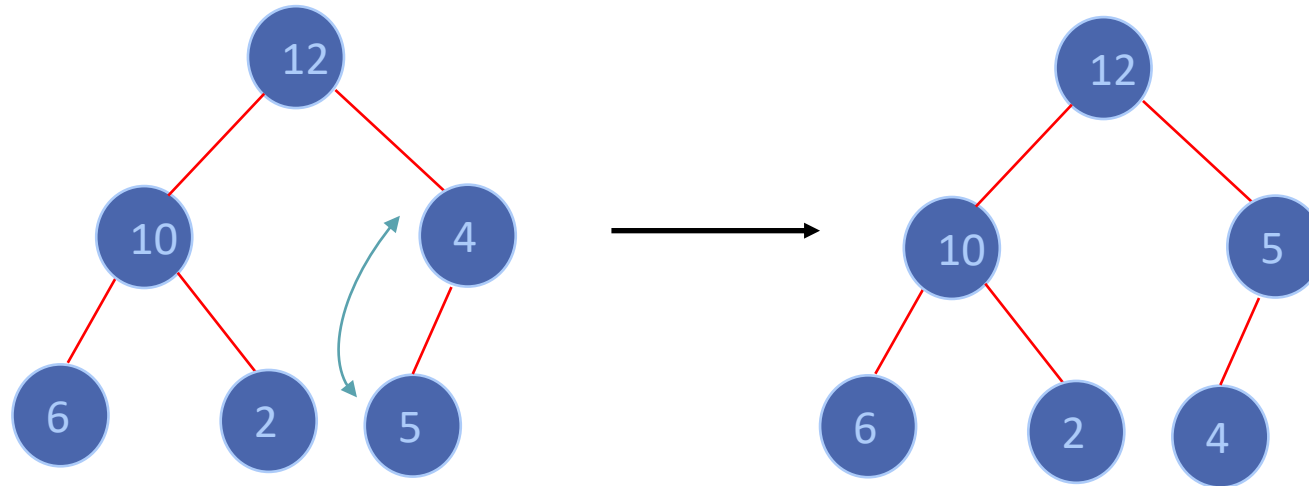
Stage 1: Heap Construction (Bottom-up)

Construct a max heap for the list 4, 6, 12, 10, 2, 5

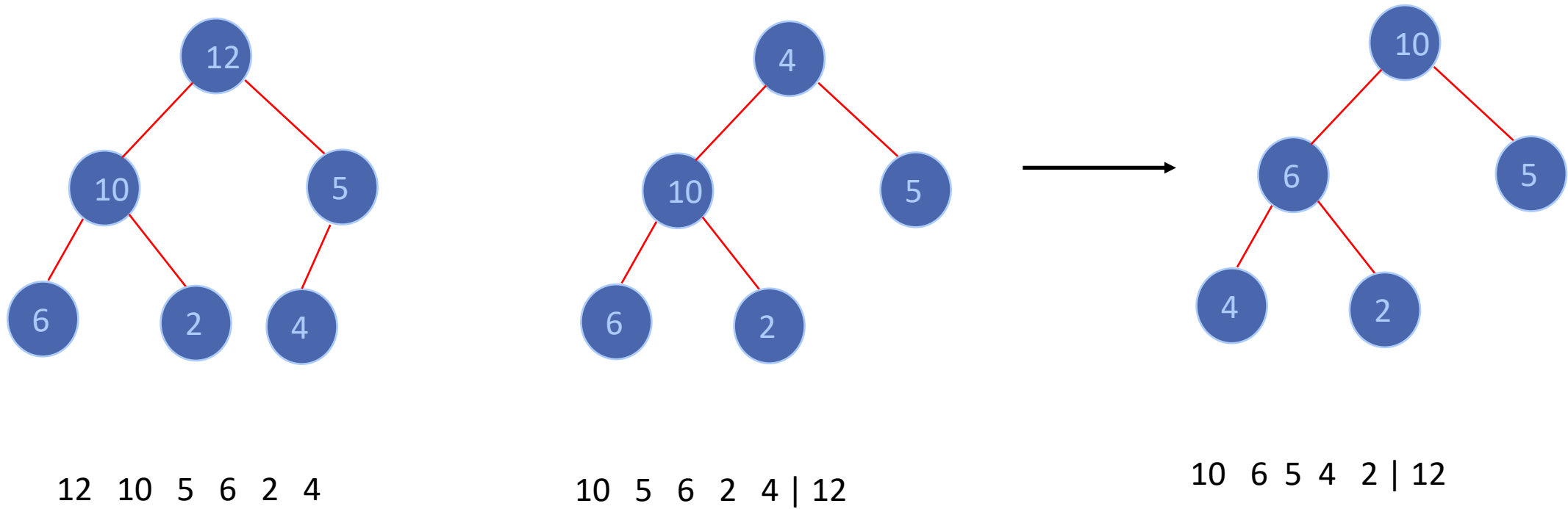


Stage 1: Heap Construction (Bottom-up)

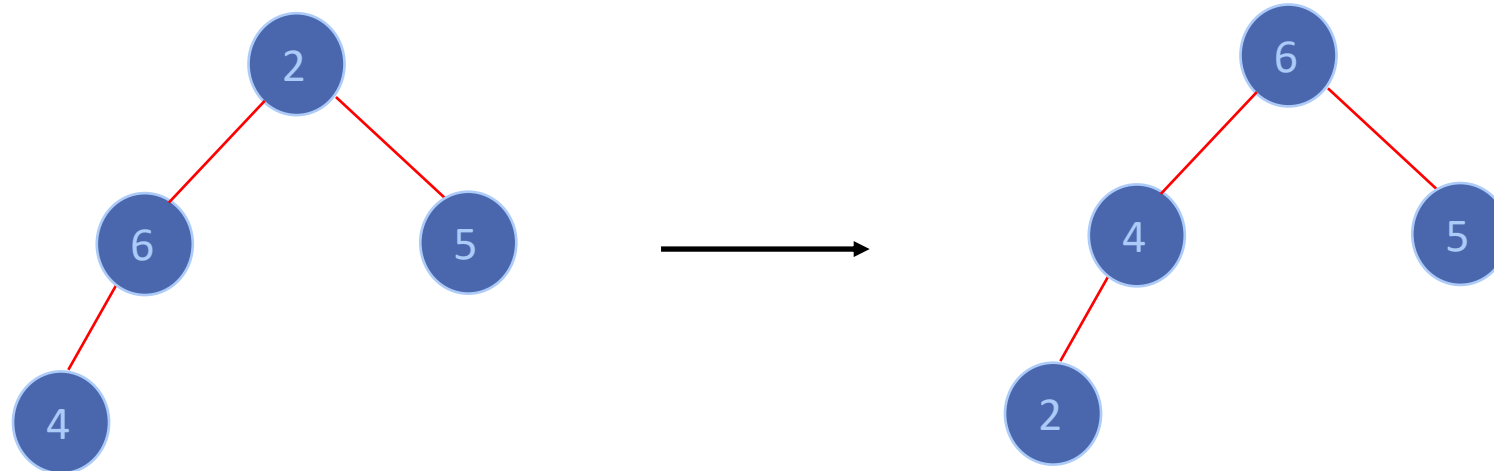
Construct a max heap for the list 4, 6, 12, 10, 2, 5



Stage 2: Root/Max Removal



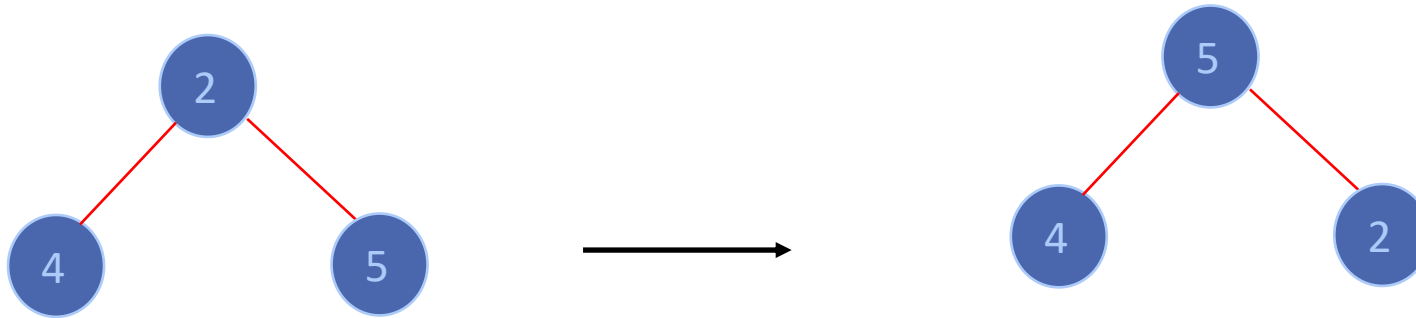
Stage 2: Root/Max Removal



2 6 5 4 | 10 12

6 4 5 2 | 10 12

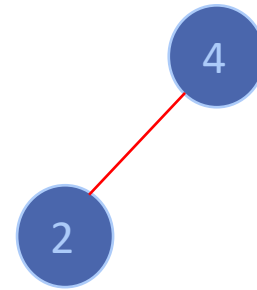
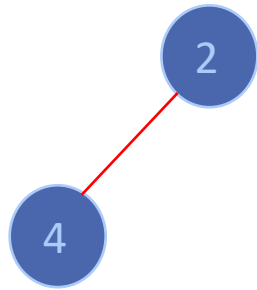
Stage 2: Root/Max Removal



2 4 5 | 6 10 12

5 4 2 | 6 10 12

Stage 2: Root/Max Removal



2 4 | 5 6 10 12

4 2 | 5 6 10 12

Stage 2: Root/Max Removal

2



2 4 5 6 10 12

2 | 4 5 6 10 12

Analysis of Heapsort

Stage 1: Build heap for a given list of n keys

worst-case

$$C_{worst}(n) = \sum_{i=0}^{h-1} \sum_{\text{level } i \text{ keys}} 2(h-i) = \sum_{i=0}^{h-1} 2(h-i)2^i = 2(n - \log_2(n+1)) \in \Theta(n)$$

Analysis of Heapsort

Stage 2: Repeat operation of root removal $n-1$ times (fix heap).

- We have to investigate just the time efficiency of the second stage.
- For the number of key comparisons, $C(n)$, needed for eliminating the root keys from the heaps of diminishing sizes from n to 2 , we get the following inequality:

$$C(n) \leq 2\lfloor \log_2(n-1) \rfloor + 2\lfloor \log_2(n-2) \rfloor + \cdots + 2\lfloor \log_2 1 \rfloor \leq 2 \sum_{i=1}^{n-1} \log_2 i$$

$$\leq 2 \sum_{i=1}^{n-1} \log_2(n-1) = 2(n-1) \log_2(n-1) \leq 2n \log_2 n.$$

$$C(n) \in O(n \log n)$$

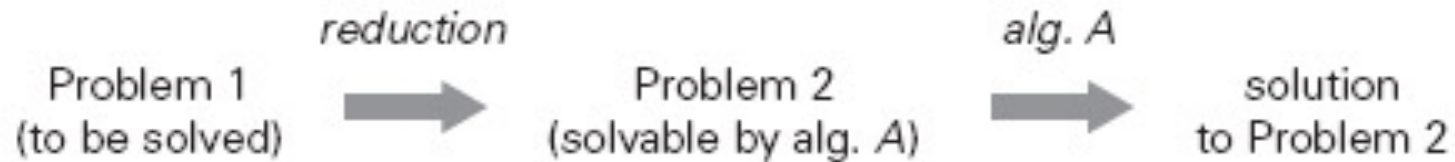
Analysis of Heapsort

- For both stages, we get $O(n) + O(n \log n) = O(n \log n)$.
- A more detailed analysis shows that the time efficiency of heapsort is, in fact, in $\Theta(n \log n)$ in both the worst and average cases.
- Thus, heapsort's time efficiency falls in the same class as that of **merge sort**.
- Unlike the latter, heapsort is in-place, i.e., it **does not require any extra storage**.
- Timing experiments on random files show that heapsort runs **more slowly than quick sort** but can be competitive with **merge sort**.

Problem Reduction

Problem Reduction

- This variation of transform-and-conquer solves a problem by transforming it into **different problem** for which an algorithm is **already available**.



- To be of practical value, **the combined time of the transformation and solving the other problem** should be **smaller** than solving the problem as given by another method.

Examples of Solving Problems by Reduction

- Computing $\text{lcm}(m, n)$ (the least common multiple) via computing $\text{gcd}(m, n)$ (the greatest common divisor)
- Transforming a maximization problem to a minimization problem and vice versa (also, min-heap construction)
- Counting number of paths of length n in a graph by raising the graph's adjacency matrix to the n -th power
- Linear programming
- Reduction to graph problems (e.g., solving puzzles via state-space graphs)

Computing The Least Common Multiple

- The **least common multiple** of two positive integers m and n , denoted $\text{lcm}(m, n)$, is defined as the **smallest integer that is divisible** by both m and n .

- For example,

$$\text{lcm}(24, 60) = 120,$$

$$\text{lcm}(11, 5) = 55.$$

- Given the **prime factorizations** of m and n , compute the **product** of all the common prime factors of m and n , all the prime factors of m that are not in n , and all the prime factors of n that are not in m .

- For example,

$$24 = 2 \cdot 2 \cdot 2 \cdot 3,$$

$$60 = 2 \cdot 2 \cdot 3 \cdot 5,$$

$$\text{lcm}(24, 60) = (2 \cdot 2 \cdot 3) \cdot 2 \cdot 5 = 120.$$

- As a computational procedure, this algorithm has the **same drawbacks** as the middle-school algorithm for computing the **greatest common divisor**.
- It is **inefficient** and requires a **list of consecutive primes**.

Computing The Least Common Multiple (Cont.)

- A much more efficient algorithm for computing the least common multiple can be devised by using problem reduction.
- There is a very efficient algorithm (Euclid's algorithm) for finding the greatest common divisor, which is a product of all the common prime factors of m and n .
- Can we find a formula relating $\text{lcm}(m, n)$ and $\text{gcd}(m, n)$?
- It is not difficult to see that the product of $\text{lcm}(m, n)$ and $\text{gcd}(m, n)$ includes every factor of m and n exactly once and hence is simply equal to the product of m and n .

- $$m \cdot n = \text{lcm}(m, n) \cdot \text{gcd}(m, n)$$

- This observation leads to the formula

$$\text{lcm}(m, n) = \frac{m \cdot n}{\text{gcd}(m, n)},$$

where $\text{gcd}(m, n)$ can be computed very efficiently by Euclid's algorithm.

Reduction of Optimization Problems

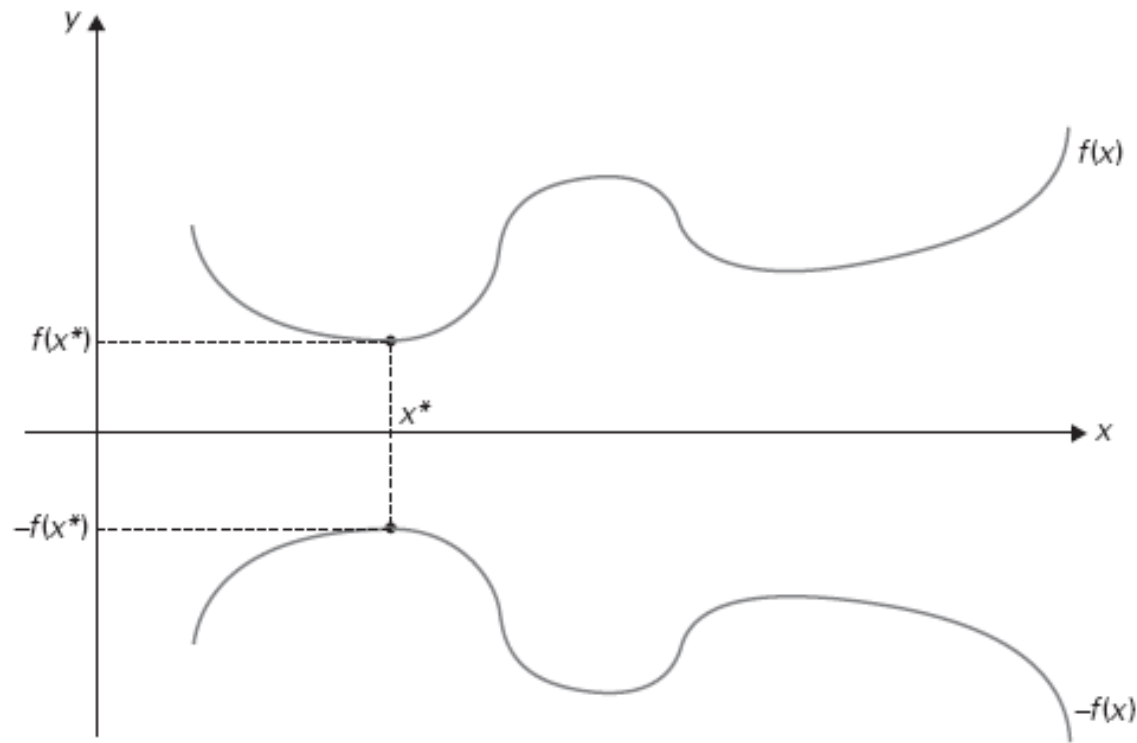
- If a problem asks to find a maximum of some function, it is said to be a ***maximization problem***;
- If it asks to find a function's minimum, it is called a ***minimization problem***.
- Suppose now that we need to find a minimum of some function $f(x)$ and we have an algorithm for function maximization.
- How can you take advantage of the latter?
- The answer lies in the simple formula :

$$\min f(x) = - \max[-f(x)].$$

- In other words, to minimize a function, we can maximize its negative instead and, to get a correct minimal value of the function itself, change the sign of the answer.

Reduction of Optimization Problems (Cont.)

- Relationship between minimization and maximization problems:



Summary

- Heaps and heapsort
- Problem Reduction
- LCM
- Optimization

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.